



# FinGuard

## 온프레미스 DevSecOps 릴리스 게이트

검사한 커밋, 승인한 이미지, 실제 배포 대상을 다시 대조하고  
잘못된 릴리스를 차단하는 Python 기반 정책 게이트

195

회귀 테스트

85.35%

테스트 커버리지

5.1.1

기준 정책 버전

100%

개인 기여도

Python

Bash

GitLab CI

Jenkins

Semgrep

Trivy

OWASP ZAP

Kubernetes

Cosign

SARIF

CycloneDX

작성자

@kwakhyun

개인 프로젝트 v0.6.4 | 기획, 아키텍처, 구현, 정책, CI/CD, 테스트, 문서화

합성 스캔 보고서와 샘플 서비스로 재현한 독립 포트폴리오입니다. 실제 운영 적용이나 규제 준수 인증을 주장하지 않습니다.

[github.com/kwakhyun/finguard-devsecops](https://github.com/kwakhyun/finguard-devsecops)

# 문제 정의와 핵심 설계

보안 도구를 연결하는 것보다 중요한 것은 검사, 승인, 배포가 같은 대상을 가리키도록 만드는 일입니다.

## 01 제각각인 보고서

도구마다 결과 형식과 심각도가 달라 같은 배포 기준을 적용하기 어렵습니다.

## 02 검사 대상 불일치

검사한 커밋, 승인한 이미지, 배포한 이미지가 다르면 개별 검사가 통과해도 안전한 릴리스가 아닙니다.

## 03 승인 자료 불일치

변경 승인, 직무 분리, 배포 허용 시간, 롤백 계획이 대상과 연결되지 않으면 감사 기록을 신뢰하기 어렵습니다.

“검증한 모든 입력이 같은 릴리스 대상을 가리킬 때만 PASS를 생성한다.”

ReleaseSubject: 배포 대상을 하나로 묶는 불변 식별자

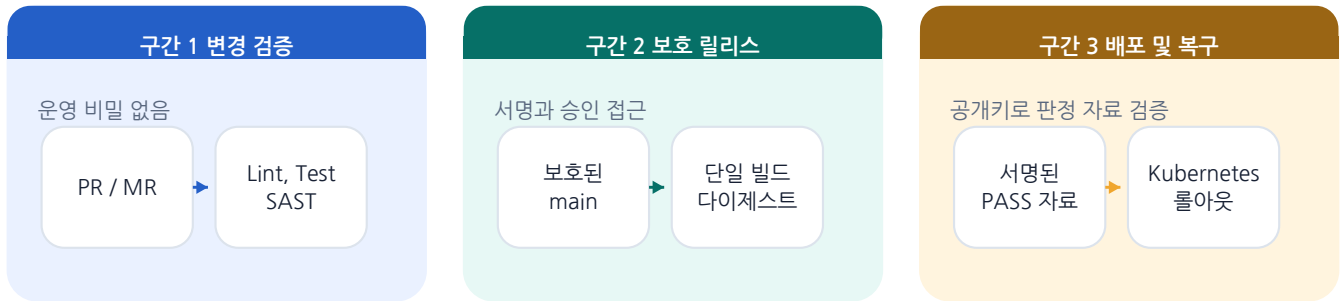


## 세 가지 설계 원칙

- 정규화: 보고서 형식을 공통 모델로 변환
- 대상 대조: 검사, 승인, 배포를 같은 대상 해시로 확인
- 오류 시 차단: 누락, 파싱 오류, 서명 실패를 취약점 0건과 구분

# 신뢰 경계를 분리한 릴리스 흐름

변경 검증과 보호 릴리스를 나누고, 각 구간에는 필요한 권한만 부여했습니다.



## 보호 릴리스 실행 순서



### 변경 검증 권한 분리

변경 코드가 승인 정보, 서명 키, Kubernetes 자격 증명에 접근하지 못하게 합니다.

### CI와 정책 로직 분리

GitLab CI와 Jenkins가 같은 Python CLI를 사용하므로 게이트 규칙이 특정 CI 제품에 종속되지 않습니다.

### 단일 빌드 이미지 재사용

레지스트리에서 받은 불변 이미지를 검사, 승인, 배포에 그대로 사용합니다.

## 구현 근거

.gitlab-ci.yml | Jenkinsfile | finguard/cli.py | tests/test\_pipeline\_contracts.py

ITSM, KMS, 보호 러너는 코드와 파이프라인 계약으로 검증한 설계 범위이며, 실제 운영 실적은 아닙니다.

## 정책 기반 품질과 보안 관리

스캐너별 다른 결과를 공통 모델로 정규화하고, 정책이 정한 필수 입력과 차단 조건을 하나의 게이트에서 평가합니다.

| 영역        | 구현 메커니즘                                   | 차단 조건 예시                        |
|-----------|-------------------------------------------|---------------------------------|
| 코드 품질     | JUnit 건수와 실패 수, 커버리지 원시 집계 교차 검증          | 실패 발생, 85% 미만, 선언값과 실제 건수 불일치   |
| SAST      | Semgrep JSON과 범용 SARIF를 공통 탐지 결과로 변환      | 차단 심각도 탐지, 보고서 누락, 스캐너 오류       |
| SCA / OSS | Trivy, CycloneDX, SPDX 표현식, VEX와 수정 버전 정보 | CRITICAL 취약점, 수정 버전 없음, 금지 라이선스 |
| DAST      | OWASP ZAP 결과와 검사 대상 URL을 이미지와 연결          | HIGH 탐지, 승인되지 않은 대상, 보고서 누락     |
| 변경 통제     | CB/SR, 승인 역할, 직무 분리, 롤백 계획과 배포 허용 시간      | 승인 부족, 같은 사람의 요청과 배포, 만료된 변경    |
| 검증 입력     | 보고서 SHA-256, 명령어와 규칙 세트 해시, 러너와 키 허용 목록   | 서명 실패, 유효 시간 초과, 커밋과 이미지 불일치    |
| 예외 정책     | 단일 탐지 지문, 독립 승인, 최대 30일, 보완 통제            | CRITICAL 예외, 범위 불일치, 만료 또는 연장   |

### merge-request.toml

변경 중 빠른 피드백

### financial-baseline.toml

로컬에서 전체 통제 재현

### financial-release.toml

보호 릴리스용 엄격 정책

### 의도한 FAIL 시나리오

CRITICAL SCA, HIGH SAST, AGPL, 테스트 실패, 직무 분리 위반

위반을 재현해 종료 코드 2를 반환합니다. 누락이나 파싱 실패도 배포를 차단합니다.

# 스캔 실행 기록과 감사 증적

결과 파일만 믿지 않고 실행 주체, 도구, 검사 대상을 확인한 뒤 입력과 판정을 함께 보존합니다.

## PASS 증적 번들

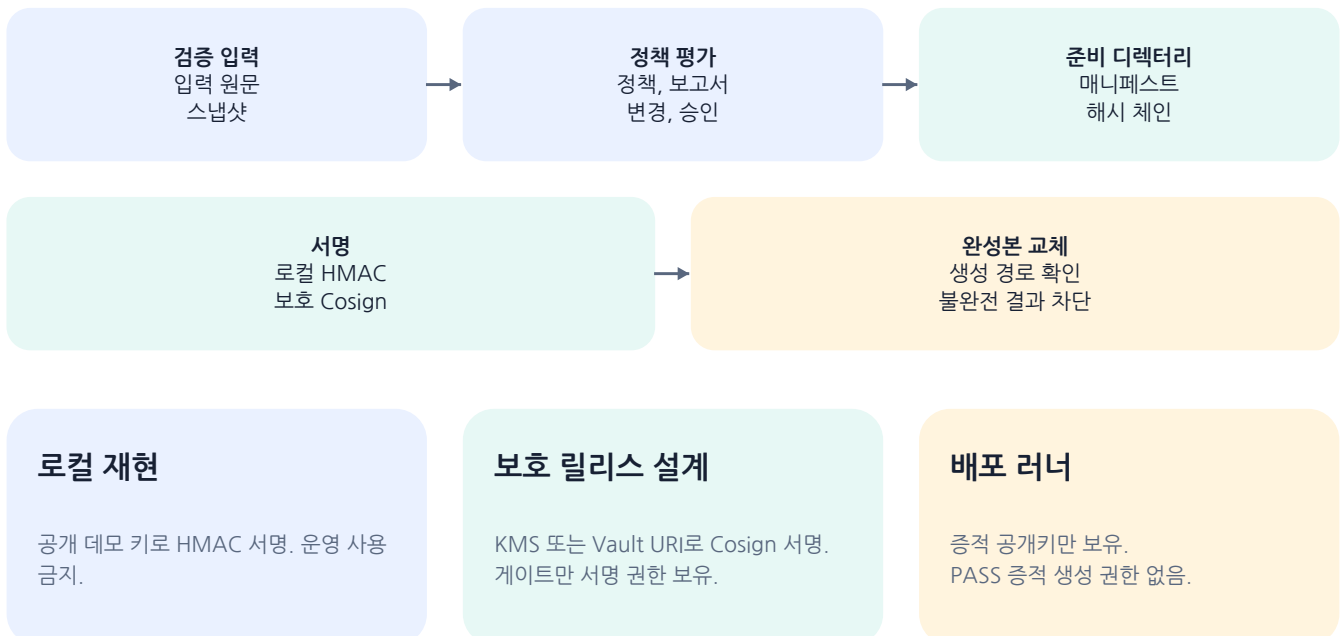
- .finguard-evidence  
FinGuard 생성 경로 표시
- manifest.json  
허용 파일과 SHA-256
- decision.json  
PASS / FAIL과 위반 사유
- audit.jsonl  
해시 체인 감사 로그
- summary.md  
사람이 읽는 요약
- inputs/  
정책, 보고서, 변경, 승인
- signature.\*  
HMAC 또는 Cosign 서명

## 스캔 실행 증명서가 확인하는 것

- 보고서 원문의 SHA-256과 스캐너 제품, 버전
- 실행한 명령어와 규칙 세트의 해시, 종료 코드
- 전체 소스 커밋, SCA와 DAST의 이미지 다이제스트
- 허용된 러너 ID, 서명 키 ID, CI 실행 정보
- 취약점 DB 해시와 갱신 시각, 보고서 신선도
- DAST 대상 URL과 승인된 상태 확인 URL의 일치

보고서를 바꾸면 서명이 무효화됩니다. 보고서가 없으면 검사 실패로 처리합니다.

## 입력을 고정하고 완성된 증적만 게시



# 증적 검증부터 자동 롤백까지

PASS 판정만 믿지 않고 서명, 정책 원문, 변경, 이미지, 배포 위치를 실행 직전에 다시 대조합니다.

## 01 사전 검증

- 증적 서명과 매니페스트
- 정책 ID, 버전, 원문 SHA-256
- 증적 생성 시각과 배포 허용 시간
- 배포 위치와 이미지 다이제스트 일치

## 02 배포 변경

- kubectl auth can-i로 RBAC 권한 확인
- 다이제스트 참조 이미지만 허용
- 변경 ID와 증적 해시를 애너테이션에 기록
- 기존 이미지와 감사 정보 저장

## 03 검증과 기록

- 롤아웃 상태와 HTTP 상태 확인
- 배포 결과를 Cosign으로 서명
- 결과 파일과 서명 번들 덮어쓰기 금지
- 실패 결과도 감사 기록으로 보존

## 자동 롤백 경로

롤아웃 실패

스모크 테스트 실패

배포 결과 저장 실패

결과 Cosign 서명 실패

Kubernetes를 변경한 뒤 실패하면 배포 직전에 확인한 이전 불변 이미지 다이제스트와 기존 FinGuard 감사 애너테이션을 복원합니다.

## 온프레미스 적용 설계

- BuildKit 또는 Podman으로 루트 권한 없이 한 번만 빌드
- 도구와 인프라 이미지를 다이제스트로 고정
- SonarQube와 PostgreSQL 이미지는 내부 레지스트리 반입 후 서명 검증
- 배포 러너에는 공개키만 두고 서명 권한은 분리

## 개발자와 운영자에게 제공하는 결과

- GitLab Code Quality JSON과 SARIF 내보내기
- 적용 전에 후보 정책과 기존 정책을 비교한 결과
- 판정, 탐지, 예외, VEX, OSS, 승인 지표를 Prometheus 형식으로 출력
- 로컬 내장 스캐너로 외부 보안 서버 없이 빠른 피드백

# 검증 결과와 제출 범위

재현 명령, 테스트 결과, 공개 CI와 검증하지 않은 범위를 함께 제시합니다.

195

테스트 통과

85.35%

핵심 패키지 커버리지

0

Ruff 위반

0

Mypy 오류

## 검증한 범위

- GitHub Actions에서 품질 검사와 E2E 데모 실행
- Semgrep, Trivy, OWASP ZAP을 공개 CI에서 실제 실행
- Cosign과 kubectl 어댑터, 하위 프로세스 계약 테스트
- 변조, 누락, 서명 오류, 유효 시간 초과 회귀 테스트
- 롤아웃, 스모크 테스트, 결과 서명 실패 롤백 경로
- main 보호, 필수 CI, 강제 푸시와 브랜치 삭제 차단

## 검증하지 않은 범위

- 합성 보고서와 샘플 서비스 기반 개인 포트폴리오
- 실제 온프레미스 서버 및 Kubernetes 운영 실적은 포함하지 않음
- Coverity와 SonarQube는 SARIF 파일만 검증. 상용 서버 API는 미연동
- FOSSA 미실행. Trivy, CycloneDX, SPDX로 OSS 관리
- 실제 도입에는 ITSM과 IdP API, 워크로드 아이덴티티, WORM 저장소, SIEM 전송 필요

## 3분 재현

```
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.lock
.venv/bin/pip install --no-deps -e .
make quality
./scripts/demo.sh
```

PASS 증적 서명과 재검증, 위험 변경 차단을 한 번에 확인합니다. FAIL 시나리오는 의도한 종료 코드 2를 반환합니다.

## 직무 역량과 연결되는 구체적 근거

### CI/CD 표준화

GitLab과 Jenkins에서 같은 CLI와 정책을 사용

### DevSecOps 자동화

SAST, SCA, DAST와 OSS 통제를 게이트에 통합

### 정책 기반 품질 관리

품질 기준과 승인 규칙을 TOML로 관리

### 배포 안전성

불변 이미지, RBAC, 서명, 자동 롤백

[저장소 및 코드](#) | [GitHub Actions 실행 이력](#) | [@kwakhyun](#)